

DEVWORKS

RALPH OPERATIONS GUIDE

Guia prático para planejar, executar e acompanhar trabalho assistido por agentes em projetos existentes, produção e projetos novos.

PRINCÍPIO OPERACIONAL

O Ralph não substitui a revisão humana. Ele executa um plano explícito, em sessões novas, e só aceita uma fase quando os quatro gates estiverem verdes.

TRÊS CAMINHOS

CHECK	AÇÃO
01	Ajustar um sistema existente sem ampliar o escopo.
02	Implementar uma funcionalidade em um projeto em produção com rollout controlado.
03	Criar um projeto do zero a partir de contexto, especificação e fases.

Versão operacional - DEVWORKS Harness

COMECE AQUI

1. Modelo mental e guardrails

O Harness separa descoberta, planejamento, contexto, execução e observação. Cada etapa deixa um artefato revisável; o próximo passo só consome o que foi explicitamente aceito.

CHECK	AÇÃO
INIT	Constrói <code>.spec/init/*</code> para um projeto novo ou pouco documentado.
PLAN	Cria <code>SPEC.md</code> , <code>PLAN.md</code> e <code>PHASES.md</code> para uma feature com slug.
CONTEXT	Mantém <code>AGENTS.md</code> , <code>CLAUDE.md</code> e <code>docs/agents/*</code> baseados no código real.
RALPH	Executa <code>PHASES.md</code> , abre uma sessão nova por fase e aplica G0, G1, G2 e G3.
MONITOR	Mostra estado e logs; não inicia, cancela, pausa ou retoma runs.

REGRA DE OURO

Antes de qualquer run: confirme repositório, branch, arquivo de fases e comandos de teste. Ralph preserva as mudanças para revisão e commit manual.

Os quatro engines têm o mesmo contrato de saída. Escolha um engine por disponibilidade e autenticação; não reescreva o plano apenas para trocar de fornecedor.

```
scripts/install-platform.sh --platform codex --scope project
./scripts/ralph.sh --engine codex .spec/features/meu-ajuste/PHASES.md
```

SEM HARNESS INSTALADO

2. Projeto existente: comandos exatos

Este é o caminho para quando o repositório contém apenas os arquivos do sistema. Mantenha o DEVWORKS Harness em outro diretório e execute os comandos abaixo na raiz do projeto-alvo.

```
cd /caminho/do/meu-projeto
git switch -c feat/minha-funcionalidade
codex login
codex --version
/caminho/devworks-harness/scripts/install-platform.sh \
--platform codex --scope project
```

CHECK	AÇÃO
cd	Entra na raiz correta; Ralph e os Skills usarão este repositório.
switch	Cria uma branch isolada, pronta para revisão e rollback.
login	Abre o fluxo oficial de autenticação da CLI; não grava token no repositório.
version	Confirma que a CLI do agente está instalada e responde normalmente.
install	Copia somente os Skills para <code>.agents/skills/</code> do projeto.

No Codex, abra esse projeto e envie primeiro:

Use o Skill `ai-context`. Analise o código existente e gere o contexto do projeto. Não implemente nenhuma funcionalidade.

Depois, descreva a funcionalidade para o Skill `plan`. Inclua objetivo, regras de negócio, perfis afetados, critérios de aceite, limites e o comando de testes. Não peça implementação nessa etapa.

```
/caminho/devworks-harness/scripts/ralph.sh --engine codex \
--test-cmd 'php artisan test' .spec/features//PHASES.md
/caminho/devworks-harness/scripts/ralph-web.sh .
```

REVISÃO MANUAL

O Ralph não faz `stage`, `commit`, `reset` nem altera exclusões do Git. Ao fim do run, revise o diff acumulado e faça o `commit` manualmente.

MENOR MUDANÇA SEGURA

3. Caso 1 - Ajuste em sistema existente

Use este caminho para correções, compatibilidade, comportamento inesperado ou uma mudança pequena e conhecida. A prioridade é preservar regras existentes e limitar o diff.

Passo a passo

CHECK	AÇÃO
1	Reproduza o problema e registre o comportamento atual, o esperado e a evidência disponível.
2	Atualize contexto se ele estiver ausente ou desatualizado: rode `ai-context` e revise a saída antes de planejar.
3	Crie uma feature curta via `plan`, com tarefas pequenas, arquivos prováveis e critérios de aceitação verificáveis.
4	Revise `PHASES.md`: garanta que não inclui refactor, migration ou alteração de regra fora do pedido.
5	Crie branch de trabalho e execute Ralph com o comando de teste real; mudanças existentes não bloqueiam o run.
6	Revise o diff acumulado, faça teste manual do cenário e só então faça o commit, merge ou deploy.
<pre>git switch -c fix/ajuste-relatado ./scripts/ralph.sh --engine codex --test-cmd 'php artisan test' \ .spec/features/ajuste-relatado/PHASES.md</pre>	
NÃO PULE Gate 1 só observa se houve escrita. Gate 3 é quem confirma task por task. Uma fase sem diff pode estar correta, mas somente se os gates finais sustentarem isso.	

PLANEJE O ROLLOUT

4. Caso 2 - Nova funcionalidade em produção

Em produção, a execução deve ser progressiva e reversível. O plano precisa explicitar compatibilidade, dados existentes, observabilidade, feature flags quando aplicáveis e rollback.

Antes do planejamento

CHECK	AÇÃO
A	Defina objetivo, usuários afetados, critérios de aceite e o que não pode mudar.
B	Mapeie contrato de API, permissões, jobs, filas, migrations, índices e integrações.
C	Defina estratégia de rollout: flag, deploy gradual, migration aditiva, métrica de saúde e rollback.

Estrutura recomendada de fases

CHECK	AÇÃO
F1	Base compatível: schema aditivo, contrato, permissão e testes de regressão.
F2	Comportamento novo protegido por configuração/flag e observabilidade.
F3	Interface, fluxo final e validação de acessibilidade.
F4	Testes integrados, checklist de deploy, rollback e documentação operacional.

DADOS E PRODUÇÃO

Não deixe o agente decidir migração destrutiva, backfill em massa ou corte de compatibilidade. Essas decisões devem constar no PHASES.md com consulta de conferência, janela de execução e plano de retorno.

```
./scripts/ralph.sh --engine claude --max-cycles 2 \  
--test-cmd './vendor/bin/sail test' \  
.spec/features/nova-funcionalidade/PHASES.md
```

DO ZERO A UM PLANO EXECUTÁVEL

5. Caso 3 - Projeto novo

Para um projeto novo, não comece pelo Ralph. Primeiro construa uma cadeia de documentos que torne decisões de negócio e arquitetura verificáveis.

CHECK	AÇÃO
1	Instale o Skill da plataforma escolhida e inicialize o repositório Git.
2	Execute `init` até revisar descrição do projeto, histórias, dados e fases iniciais em `.spec/init`.
3	Acrescente referências de interface em `.spec/init/design` quando houver telas, componentes ou inspiração visual.
4	Revise `.spec/init/project-phases.md`: cada `## Phase N:` deve ter tasks, arquivos e testes possíveis.
5	Rode a primeira fase em uma branch isolada e trate o resultado como uma revisão de arquitetura, não só como código gerado.
6	Depois de código real existir, rode `ai-context` para documentar a arquitetura observada.
<pre>git init scripts/install-platform.sh --platform opencode --scope project # Use o workflow init na interface da sua plataforma ./scripts/ralph.sh --engine opencode .spec/init/project-phases.md</pre>	
DEFINIÇÃO DE PRONTO	
Um projeto novo não está pronto quando compila. Ele está pronto quando os fluxos essenciais, testes, configuração, segurança, deploy e operação foram assumidos explicitamente no plano.	

O QUE O RALPH FAZ

6. Execução e acompanhamento

O Ralph abre uma sessão nova em cada fase e em cada ciclo de correção. Ele não usa exit code do agente como prova final. Os gates são mecânicos:

CHECK	AÇÃO
G0	A CLI terminou conforme o contrato do adaptador.
G1	A sessão alterou a árvore? É sinal, não veredito.
G2	A suíte de testes configurada passou fora da sessão do agente.
G3	Um verificador read-only confirma `TASK n: DONE/INCOMPLETE`. Este é o veredito final.

Acompanhe pelo TUI ou pelo monitor web

```
./scripts/ralph-watch.sh /caminho/do/projeto
./scripts/ralph-web.sh /caminho/do/projeto
# abre http://127.0.0.1:7331
```

O monitor mostra projeto, engine, run, PID, duração, progresso, fase, ciclo, gate, tabela hierárquica, logs recentes e histórico. Ele lê somente `.phases/`; não expõe a execução à rede e não tem botões de controle.

A estrutura de estado permite interromper o terminal sem apagar diagnóstico. `runs//` mantém logs e prompts; `progress/.progress` só retoma um documento de fases idêntico.

ANTES, DURANTE E DEPOIS

7. Checklist de decisão e operação

Antes de executar

CHECK	AÇÃO
☐	Repositório, branch e `PHASES.md` corretos.
☐	Alterações existentes foram identificadas; Ralph não as inclui em commits nem altera o índice.
☐	CLI autenticada; engine escolhido explicitamente.
☐	Comando de testes real definido; ambiente necessário disponível.
☐	Dados, migrations, integrações e rollback revisados quando houver produção.

Durante

CHECK	AÇÃO
☐	Observe G0-G3 e trate limite de uso como espera, não como falha funcional.
☐	Não altere o plano enquanto o run está ativo. Pare e crie um novo plano se o escopo mudou.
☐	Use logs e prompts do run para investigar; não exponha tokens ou dados sensíveis.

Depois

CHECK	AÇÃO
☐	Revise o diff completo e faça o commit manualmente, quando desejado.
☐	Execute validação manual dos critérios de aceite.
☐	Para produção: valide métricas, logs, rollback e comunicação operacional.

COMANDO DE VALIDAÇÃO DO HARNESS

```
scripts/check-shell.sh
scripts/check-skill-drift.sh
python3 -m unittest tests/test_dashboard.py
git diff --check
```